Danmarks
Tekniske
Universitet

02244 - Logic for Security
Project on Security Protocols

Authors

Stefano Licinio - s232450
Andrej Jurco - s243889
Sumaya Ahmed Promi - s240007

March 16, 2025

Contents

Introduction - Sumaya Ahmed Promi **1**

1 Signup protocol - Stefano Licinio **1**
1.1 Outline of the design . . . 1
1.1.1 Agents and initial knowledge . . . 1
1.1.2 Security properties to establish . . . 2
1.2 AnB formalization . . . 2
1.2.1 Design decisions and development . . . 2
1.2.2 Modeling in AnB . . . 3

2 Upload Offer Protocol - Andrej Jurco **5**
2.1 Outline of the design . . . 5
2.1.1 Agents and initial knowledge . . . 5
2.1.2 Security properties to establish . . . 5
2.2 AnB formalization . . . 6
2.2.1 Design decisions and development . . . 6
2.2.2 Modeling in AnB . . . 6

3 Search Protocol - Sumaya Ahmed Promi **7**
3.1 Outline of the design . . . 7
3.1.1 Agents and initial knowledge . . . 7
3.1.2 Security properties to establish . . . 8
3.2 AnB formalization . . . 8
3.2.1 Design decisions and development . . . 8
3.2.2 Modeling in *AnB* . . . 8

4 Purchase Protocol - Stefano Licinio **10**
4.1 Outline of the design . . . 10
4.1.1 Agents and initial knowledge . . . 10
4.1.2 Security properties to establish . . . 11
4.2 AnB formalization . . . 11
4.2.1 Design decisions and development . . . 11
4.2.2 Modeling in AnB . . . 11

References **14**

Introduction - Sumaya Ahmed Promi

To establish a secure and privacy-preserving online marketplace for books, this project focuses on designing security protocols for an independent e-book platform. The goal is to provide *User* with a secure experience while signing up, searching for books, uploading offers and making purchases. Given the increasing threats to online privacy and data integrity, it is crucial to develop cryptographic solutions that ensure authentication, confidentiality and integrity in all transactions.

This report presents the design and formal verification of four essential protocols:

1. **Signup Protocol:** Establishes a secure cryptographic relationship between a new *User* and the *Website* through a trusted third party *(server)*.
2. **Upload Offer Protocol:** Allows authenticated *User* to securely *(Upload)* book-list for sale while ensuring *authenticity and integrity*.
3. **Search Protocol:** Enables authenticated *User* to *(Search)* for books privately, preventing unauthorized access and query manipulation.
4. **Purchase Protocol:** Facilitates secure transactions between *Buyers* and *Sellers* while maintaining *authentication and privacy*.

Each protocol is designed using formal security models, including *AnB* formalization. It verifies correctness and security against potential attacks. The implementation considers authenticated *User* while preserving security guarantees. This report details the development, design choices and verification process of these protocols, ensuring they meet the security requirements necessary for a trustworthy marketplace.

1 Signup protocol - Stefano Licinio

1.1 Outline of the design

1.1.1 Agents and initial knowledge

According to the assignment text, a new user wants to register from scratch to the website and an honest party is going to establish cryptographic relationship between the two of them. Moreover, every user has a password with the honest party and everyone has a public/private key pair. However, in the beginning, the user and the website only knows the public key of the honest party and the latter only knows the identity of the website (the honest party doesn't know the identity of any user).

For the sake of simplicity, we will call the user "A", the website "B" and the honest party "s".

The cryptographic keys used in the protocol are:

- a shared key between s and A and s and B;
- a public/private key pair for every agent involved;
- a symmetric key that will be considered as the fresh cryptographic relationship between A and B, freshly generated by s during the protocol actuation.

The core messages that proved valuable to fulfill the requests of the assignment are:

- Two nonces NA and NB , since they hold time information to prove the freshness of the messages exchanged;
- A symmetric key KAB that works as an established way of communicating securely between A and B.

1.1.2 Security properties to establish

KAB needs to be a secret between A and B only because it is the cryptographic relationship that s freshly establishes between them. NA and NB needs to be considered a secret between A and B, since they hold information about the session in which KAB is shared. Altogether NA, NB and KAB are used by B to authenticate A.

1.2 AnB formalization

The whole development of the signup protocol was made before the changes made to the assignment request (user and website didn't have a public/private key pair). However, the solution was scalable with the changes and it didn't affect it that much. I will label "old_" every protocol made before the changes on the assignment's text.

1.2.1 Design decisions and development

The first protocol I wrote to address the signup request was very simple: A saying to s that it wanted to talk with B, s sending A's identity to B together with a fresh KAB, B simply sending back to A a nonce NA through s, s gives back to A NA and KAB, A sending directly to B NA encrypted with KAB to be authenticated.
However, OFMC finds an attack and finds a way to leak the secret KAB by using two sessions of the same protocol.
Since the attack involves the use of two sessions to leak a secret, the fix was to exchange the first nonce in plaintext as the first step of the protocol and then write a better way to encrypt information by using in a proper way the shared key given.
After the edits made to the assignment request, I've made some minor changes to better fit the new requests and, after polishing everything, came to the final version.

1.2.2 Modeling in AnB

The first version is the protocol made before the adjustments and before the final polishing. The second version is the one that is being used.

The simplifications made where:

- KAB functions as a shared cryptographic secret (One Time Token/Password shared given by the honest party to A which let him talk with B);
- s doesn't authenticate A or B on specific parts of knowledge because it will give the attacker an opportunity for replay attacks (a solution has yet to be found).
- It is enough for us that assuming that KAB, NA and NB are secret between A and B and that they are used as an authentication mean is enough to prove that everyone in the communication is honestly authenticated.

Edit on the shared cryptographic knowledge Clarification made by the professor on the specifications of the signup protocol:
"Suppose you establish a key K in the signup between two roles Alice and Bob. Then the other protocols can have in the initial knowledge of roles Alice and Bob (if these roles correspond to the ones in the signup protocol) a fixed long-term key. It is of course not allowed to put a variable K in the initial knowledge of roles, but you need for that a function, say bookkey(Alice,Bob). I recommend to use a function like bookkey that is different from all other functions you have used in the signup protocol."
KAB in the old signup protocol has been changed to bookkey(A,B) accordingly.

Old_Signup.AnB

```
Protocol: Signup
Types:
        Agent A,B,s;
        Number NA,NB;
        Symmetric_key KAB;
        Function sk, pk;

Knowledge:
        A: A, B, s, sk(A,s), pk(s);
        B: A, B, s, sk(B,s), pk(s);
        s: A, B, s, sk(A,s), sk(B,s), pk(s), inv(pk(s));

Actions:
        B -> A: NB
```

```
        A -> s: { {| A, B, NA, NB |}sk(A,s), A, B, NA, NB }pk(s)
        s -> B: { {| {| A, B, NA, NB |}sk(A,s), KAB, NA, NB |}sk(B,s) }inv(
            pk(s))
        B -> s: { {| {| A, B, NA, NB |}sk(A,s), {| A, B, NA, NB |}sk(B,s) |}
            sk(B,s) }pk(s)
        s -> A: { {| {| A, B, NA, NB |}sk(A,s), {| A, B, NA, NB |}sk(B,s),
            KAB, NA, NB |}sk(A,s) }inv(pk(s))
        A -> B: {| {| A, B, NA, NB |}sk(B,s) |}KAB

Goals:
        KAB secret between A, B;
        B authenticates A on NA;
```

Signup.AnB

```
Protocol: Signup
Types:
        Agent A, B, s;
        Number NA, NB;
        Function sk, pk, bookkey;
Knowledge:
        A: A, B, s, sk(A,s), pk(A), inv(pk(A)), pk(s);
        B: A, B, s, sk(B,s), pk(B), inv(pk(B)), pk(s);
        s: A, B, s, sk(A,s), sk(B,s), pk(s), inv(pk(s)), pk(B), bookkey(A,B)
            ;
Actions:
        A -> B: A
        B -> A: NB
        A -> s: { {| A, B, NA, NB, pk(A) |}sk(A,s), A, B, NA, NB }pk(s)
        s -> B: { {| {| A, B, NA, NB, pk(A)|}sk(A,s), A, B, pk(A), bookkey(A
            ,B), NA, NB |}sk(B,s) }inv(pk(s))
        B -> s: { {| {| A, B, NA, NB, pk(A) |}sk(A,s), {| A, B, NA, NB, pk(A
            )|}sk(B,s) |}inv(pk(B)) }pk(s)
        s -> A: { { {| {| A, B, NA, NB, pk(A) |}sk(A,s), {| A, B, NA, NB, pk
            (A) |}sk(B,s), A, B, pk(A), bookkey(A,B), NA, NB |}sk(A,s)}pk(A)
            }inv(pk(s))
        A -> B: {| {| A, B, NA, NB, pk(A) |}sk(B,s) |}bookkey(A,B)
Goals:
        bookkey(A,B) secret between A, B;
        B authenticates A on NA, NB, bookkey(A,B);
        NA, NB secret between A, B;
```

2 Upload Offer Protocol - Andrej Jurco

2.1 Outline of the design

The Upload Offer protocol allows a registered user to securely submit an offer to website. The main objective is to ensure that the website can verify the identity of the user and guarantee the integrity of the uploaded offer.

To achieve this, the protocol must establish a secure communication channel between the user and the website, preventing unauthorized access or manipulation. Since the offer details (such as book title, author, price, and description) are considered an arbitrary chunk of information, the protocol does not define a specific data structure for them.

2.1.1 Agents and initial knowledge

The protocol involves two main participants:

- **U (User)**: The registered seller who wants to upload an offer.
- **W (Website)**: The marketplace that verifies and stores the offer.

Each agent has the following cryptographic knowledge:

- A symmetric key *bookkey(U,W)* that was established during the sign-up for secure communication.
- Nonces NU and NW to ensure message freshness and protect against replay attacks.
- Encrypted offer information to prevent unauthorized modification during transmission.

2.1.2 Security properties to establish

The Upload Offer protocol must ensure the following properties:

- **Confidentiality:** The offer details remain secret between the user and the website.
- **Message Freshness:** The use of nonces prevents replay attacks.
- **Authentication:** The website must verify that the offer originates from the correct user.
- **Data Integrity:** The offer cannot be modified during transmission.

2.2 AnB formalization

2.2.1 Design decisions and development

Initially, the protocol only transmitted the offer directly, but it was vulnerable to replay attacks. To not allow this, nonces NU and NW were introduced. These ensure that the request and response are fresh, preventing attackers from resending previous messages.

2.2.2 Modeling in AnB

Below is the final AnB model of the Upload Offer protocol, incorporating security features such as encryption, authentication, and nonce usage.

```
Protocol: UploadOffer
Types:
        Agent U,W;
        Number Offer,NU,NW;
        Function bookkey;
Knowledge:
        U: U,W,bookkey(U,W);
        W: U,W,bookkey(U,W);
Actions:
        U -> W: {|NU,U, Offer |}bookkey(U,W)
        W -> U: {|NW, NU |}bookkey(U,W)
        U -> W: {|NW|}bookkey(U,W)
Goals:
        Offer secret between U, W;
        NU secret between U, W;
        W authenticates U on NU;
        W authenticates U on Offer;
```

3 Search Protocol - Sumaya Ahmed Promi

3.1 Outline of the design

The *Search* protocol ensures that users can *securely search* on the e-book marketplace while maintaining *privacy and preventing unauthorized manipulation*. It supports *authenticated users*, ensuring the *confidentiality* of search terms and the *integrity* of results.

- **Authenticated Search:** An authenticated user has a *cryptographic relationship with the website* established during the signup process. This ensures *integrity and confidentiality* in *Search* queries.

3.1.1 Agents and initial knowledge

In the *Search* protocol, a *User* interacts with the *Website* to search for books, while an honest third-party *server* ensures security and privacy. The goal is to allow users to search for books securely without leaking search queries to unauthorized entities.

For simplicity, we define the following agents:

1. **User:** The user *(authenticated)* searching for books.
2. **Website:** The e-book *website* - manages book listings, responds to search queries.
3. **server:** The trusted server that helps establish cryptographic relationships.

The cryptographic elements involved include:

1. A symmetric session key *KAB* between *User* and *Website*, established during signup.
2. *Public/private key pairs* for all agents.
3. Nonces *NA* and *NB* to ensure freshness and prevent replay attacks.
4. A *search* query, securely transmitted from *User* to *Website*.

Each agent has the following initial knowledge:

1. Every agent possesses a *public/private key pair*.
2. The honest party *server* has a *public/private key pair*, and everyone knows its public key.
3. The *Website* has a *public/private key pair*, known only to *server* initially and the cryptographic relationships *(KAB)* established during signup.
4. An authenticated *User* has a *shared session key* with the *Website*, which was created during signup.

3.1.2 Security properties to establish

The *Search* protocol must ensure the following:

1. **Authenticated Queries:** For authenticated *User*, the *Website* should verify their identity before executing the *Search*.
2. **Confidentiality of the Search Term:** The *Search* query must remain private between *User* and *Website*. The protocol ensures that no third party or any potential attackers can intercept or read it.
3. **Query Privacy:** No external party should be able to view or modify the *Search* query.
4. **Response Integrity:** *Search-Result* should not be altered or manipulated by an attacker.

3.2 AnB formalization

In this section, we discuss a structured representation of the *Search* protocol, ensuring its correctness and security. Here, we define the roles of agents, cryptographic elements and message exchanges necessary to achieve *authentication, confidentiality and integrity*. We also describe the key design choices, development iterations and the final *AnB* model used for verification.

3.2.1 Design decisions and development

The design of the *Search* protocol considers *authenticated* users only. We designed the search protocol to ensure strong *privacy and integrity*.

Key design decisions:

1. For *authenticated User*, the search query is encrypted using the *session key (KAB)*.
2. Nonces *NA* and *NB* are incorporated to prevent *replay attacks*.
3. The *Website* responds with *encrypted search results* to ensure *integrity*.

3.2.2 Modeling in *AnB*

In this section, we present the final *AnB* model of the *Search* protocol, incorporating security measures such as *encryption, authentication and nonce* usage. It ensures that the protocol meets *confidentiality, integrity and privacy* requirements while allowing *authenticated searches*. This protocol ensures that the *Search* query is securely transmitted while keeping user *privacy* intact and ensuring that their queries remain *private and secure* from manipulation.

Search.AnB

```
Protocol: Search
Types:
        Agent User, WebSite, server;
        Number NA, NB, Search, Search_Result;
        Symmetric_key KAB;
        Function sk, pk;
Knowledge:
        User:    User, WebSite, server, sk(User,server),     pk(User),
                   pk(server),  inv(pk(User));
        WebSite: User, WebSite, server, sk(WebSite,server),  pk(WebSite),
                 pk(server),  inv(pk(WebSite));
        server:  User, WebSite, server, sk(WebSite,server), sk(User,server),
               pk(server),  inv(pk(server)),   pk(WebSite);
Actions:
        WebSite -> User:   NB
        User -> server:    { {| User, WebSite, NA, NB, pk(User) |}sk(User,
           server), User, WebSite, NA, NB }pk(server)
        server -> WebSite: { {| {| User, WebSite, NA, NB, pk(User)|}sk(User,
           server), User, WebSite, pk(User), KAB, NA, NB |}sk(WebSite,server
           ) }inv(pk(server))
        WebSite -> server: { {| {| User, WebSite, NA, NB, pk(User) |}sk(User
           ,server), {| User, WebSite, NA, NB, pk(User)|}sk(WebSite,server)
           |}inv(pk(WebSite)) }pk(server)
        server -> User:    { { {| {| User, WebSite, NA, NB, pk(User) |}sk(
           User,server), {| User, WebSite, NA, NB, pk(User) |}sk(WebSite,
           server), User, WebSite, pk(User), KAB, NA, NB |}sk(User,server)}
           pk(User) }inv(pk(server))
        User -> WebSite:   {| {| User, WebSite, NA, NB, pk(User) |}sk(
           WebSite,server) |}KAB
#Search
        WebSite -> User: {| {| User, WebSite, NA, NB, pk(User) |}sk(WebSite,
           server) |}KAB
        User -> WebSite: {| User, WebSite, Search, NA, NB, pk(User) |}KAB
        WebSite -> User: {| User, WebSite, Search_Result, NA, NB, pk(WebSite
           ) |}KAB
Goals:
        KAB secret between User, WebSite;
        WebSite authenticates User on NA, NB;
        NA, NB secret between User, WebSite;
        Search secret between User, WebSite;
        Search_Result secret between User, WebSite;
```

4 Purchase Protocol - Stefano Licinio

4.1 Outline of the design

The purchase protocol designed in this report is a simple protocol that makes a buyer and a seller sign a purchase contract in a secure way. Its design is very simple: the two users involved authenticate each other by using TLS and then exchange a contract that has to be signed by both of them using a protocol solution discussed in [1].
I would like to briefly talk about the content of this paper since it has been a real gamechanger for the whole design of this protocol and, also, further thanks the authors for both the solutions provided and the discussion about the feasibility, pros and cons of using and not using a trusted-third-party (ttp) in a contract signing procedure.

The solution provided in section 4 of [1] states that users involved in the contract agreement exchange tokens of non-repudiation as evidence of their will to sign the contract. After the exchange, the authors of [1]. After exchanging these evidences, the user that first ask for the contract to be signed, sends the contract to the ttp to store a copy of it. In the case of [1], the ttp is a blockchain transaction. However, in our implementation, the blockchain transaction is treated as a black box and thus is just represented as an honest party.

To overcome the possibility shown in the last challenge of the project, which is one part terminating the communication after having the full contract first, we can just state the the contract is valid only after its transaction on the blockchain gets validated or, in our case, when the ttp s receives it.

It has also been stated in [1] that:

> *Once the blockchain transaction Tx is in the blockchain, Alice can optionally provide the transaction identifier for Tx to Bob. We prefer not to add it to the protocol specification because it does not provide security to the proposed scheme (this message could be lost or may not be sent by Alice).*

However, since we're treating the blockchain as a blackbox, we can avoid this problem by stating that if the merchant needs the proof that the contract has been signed and has correctly uploaded on the blockchain, it can just ask to the ttp s. It is of course another relaxation made on this problem but eventually, with more time and resources in general, it is something that can be properly formalized.

4.1.1 Agents and initial knowledge

In the project specification it is stated that:

> *The purchase protocol is between a merchant who wants to sell a book, a buyer who wants to purchase this book, and possibly the website. Both buyer and seller must be registered users. It is actually desirable that the website does not need to be involved in the purchase.*

This lead us to design a protocol between only the two directly involved pats: the buyer and the seller.

Both buyer and merchant are registered users of the website, so this means that they both went through the signup protocol. From the signup protocol, we know that every user (Buyer in the purchase protocol) has a cryptographic relationship (bookkey(User,Website)) established with the website in a secure way. Given this, we can take for granted the fact that buyer can ask to the Website who's the merchant selling a specific book, without anyone knowing the book they're looking for. At the end of this procedure, we can take for granted that buyer has the knowledge of merchant (thus explicitly stated in the Knowledge section of protocol). The merchant still doesn't know who the buyer is, before it introduces itself during the first phases of the protocol.

In this protocol, besides what we have just stated, each (possibly dishonest) agent know the information needed for TLS to safely authenticate them. Regarding the ttp s, it only knows the basic information needed to accept the contract and tokens of non-repudiation.

4.1.2 Security properties to establish

That the two users gets correctly authenticated and that they establishes a safe communication channel. After establishing a safe communication channel, the contract along with the non-repudiation tokens can be safely exchanged between the users involved.

Also, the design of the protocol as in [1] satisfies fairness, timeliness, confidentiality an non-repudiation requirements.

4.2 AnB formalization

4.2.1 Design decisions and development

The purchase protocol is made of two fundamental blocks:

1. The first block is the TLS protocol, as formalized in [2];
2. The second block represents the Contract Signing protocol.

The impossibility of formalizing a blockchain transaction has been overcome by considering it as a black box in which we send the certificate and receive the transaction validation.

4.2.2 Modeling in AnB

Purchase

```
Protocol: purchase_tls

Types:
        Agent Acquirer, Seller, s;
        Number NA, NS, Sid, PA, PS, PMS;
        Function pk, hash, clientK, serverK, prf;
        Message Contract,TransactionIsValid;

Knowledge:
        Acquirer: Acquirer, pk(Acquirer), pk(s), inv(pk(Acquirer)), {Acquirer, pk(Acquirer)}inv(pk(s)), Seller, hash, clientK, serverK, prf, s;
        Seller: Seller, pk(Seller), pk(s), inv(pk(Seller)), {Seller, pk(Seller)}inv(pk(s)), hash, clientK, serverK, prf;
        s: Acquirer, Seller, pk(s), inv(pk(s)), pk(Acquirer), pk(Seller);

Actions:
#
### TLS STARTS ###
#
        Acquirer -> Seller: Acquirer, NA, Sid, PA
        Seller -> Acquirer: NS, Sid, PS, {Seller, pk(Seller)}inv(pk(s))
        Acquirer -> Seller: {Acquirer, pk(Acquirer)}inv(pk(s)), {PMS}pk(Seller), {hash(NS, Seller, PMS)}inv(pk(Acquirer)), {|hash(prf(PMS, NA, NS), Acquirer, Seller, NA, NS, Sid, PA, PS, PMS)|}clientK(NA, NS, prf(PMS, NA, NS))
        Seller -> Acquirer: {|hash(prf(PMS, NA, NS), Acquirer, Seller, NA, NS, Sid, PA, PS, PMS)|}serverK(NA, NS, prf(PMS, NA, NS))
#
### TLS ENDS ###
#
#
### CONTRACT AGREEMENT BETWEEN ACQUIRER AND SELLER ###
#
        Acquirer -> Seller: Contract,Sid,{Acquirer,pk(Acquirer)}inv(pk(s)),{Contract,Sid,{Acquirer,pk(Acquirer)}inv(pk(s))}inv(pk(Acquirer))
        Seller -> Acquirer: {Contract,Sid,{Acquirer,pk(Acquirer)}inv(pk(s))}inv(pk(Seller))
#
### SENDING CONTRACT TO TRUSTED THIRD PARTY S ###
#
        Acquirer -> s: hash(Contract,Sid,{Acquirer,pk(Acquirer)}inv(pk(s)),{Contract,Sid,{Acquirer,pk(Acquirer)}inv(pk(s))}inv(pk(Acquirer)))
```

```
            ,hash({Contract,Sid,{Acquirer,pk(Acquirer)}inv(pk(s))}inv(pk(
            Acquirer)),{Contract,Sid,{Acquirer,pk(Acquirer)}inv(pk(s))}inv(pk
            (Seller)))
        s -> Acquirer: {TransactionIsValid}inv(pk(s))

Goals:
        Seller authenticates Acquirer on prf(PMS, NA, NS)
        Acquirer authenticates Seller on prf(PMS, NA, NS)
        prf(PMS, NA, NS) secret between Acquirer, Seller
```

References

[1] Josep-Lluis Ferrer-Gomila, M. Francisca Hinarejos, Andreu-Pere Isern-DeyĂ , "A fair contract signing protocol with blockchain support," *Elsevier*, 2019-07.

[2] Sebastian Modersheim, *Protocol Security Verification Tutorial.* DTU Compute, Spring 2018.